UNIVERSITY OF SZEGED

Web application for bilingual speech recognition deployed in cloud

A Thesis submitted in partial satisfaction of the requirements
for the degree Master of Science

in

Computer Science

by

Kirill Golubev

Supervisor: Dr. Gabor Gosztolya, associate professor

2025

Copyright

Kirill Golubev, 2025

All rights reserved.

Assignment Statement / Feladatkiírás

The task of the student is to develop a cloud-based speech transcription web application. It has to contain standard user-handling functions (e.g. registering, authentication, authorization, user roles). Furthermore, it has to include the utilization of speech-to-text models for different languages, which have to be trained by the student. The models do not have to be state-of-the-art though, as the aim of the project is a proof of concept (i.e. to show that it can be done); training a more accurate model with more training data is just a question of resources afterwards.

A hallgató feladata egy felhőalapú beszédleiratozó (speech-to-text) webalkalmazás fejlesztése. Az alkalmazásnak tartalmaznia kell a szokásos felhasználó-kezelési funkciókat (regisztráció, bejelentkezés, jogosultság-kezelés). Tartalmaznia kell emellett különböző nyelvekre tanított beszédfelismerő modelleket, amelyeket a hallgatónak kell betanítania. A modellektől ugyxanakkor nem elvárás a state-of-the-art teljesítmény, csak meg kell mutatnia, hogy megvalósítható a feladat (proof of concept), valamint a hallgató képes is elvégezni; pontosabb modellek tanítása nagyobb paraméterszámú modellek több tanítóadattal finomhangolásával már csak erőforrások kérdése.

Content summary

- ***Title:***

Web application for bilingual speech recognition deployed in cloud

- ***Formulation of the task:***

Develop an application, which will be able to do Voice-to-Text operations, using a self finetuned ASR model.

- ***Solution method:***

Flask framework for the backend, JS+CSS+HTML as a static frontend for the application, which is deployed using standard or cloud architecture. WHisper model finetuning using customized script on a huggingface dataset.

- ***Tools and methods used:***

Flask, JS+CSS+HTML, Gunicorn+Nginx and Microstack. Whisper Tiny model, huggingface dataset.

- ***Achieved results:***

Model was finetuned on a small subset (as a proof of concept) achieving 33% average WER. Web-application was developed, which is able to transcribe users' microphone recordings in three different languages caching previous transcriptions.

- ***Keywords:***

Web-application, Whisper finetuning, Flask restFull API, Openstack

Table of contents / Tartalomjegyzék

Assignment Statement / Feladatkiírás 3
Content summary 4
Table of contents / Tartalomjegyzék 5
INTRODUCTION / BEVEZETÉS 6
1. Background of the topic 6
1.1. Motivation 6
1.2. Existing solutions overview 7
1.3. Research Gap 8
2. Model Fine-Tuning 9
2.1. Why Whisper? 9
2.1.1. Comparison to the alternatives 9
2.1.2. Why Whisper Tiny? 10
2.1.3. Architecture explained 10
2.2. Why Huggingface interface? 12
2.2.1. Huggingface package 12
2.2.2. Streaming mode and Caching 13
2.2.3. Mozilla Common Voice 13
2.3. Structure of finetune script 14
2.3.1. Argument parsing 14
2.3.2. Model loading 14
2.3.3. Data preparation 15
2.3.4. Training loop 15
2.4. Customizations to the finetune script 16
2.4.1. Encapsulation and changes of data preparation 16
2.4.2. GPU training 16
2.5. Dataset composition and training 17
2.6. Evaluation of the English model 19
3. Web-application 21
3.1. Why Flask? 21
3.2. Backend structure 22
3.3. MongoDB structure 26
3.4. Frontend Structure 28
3.5. UI overview 30
4. Deployment 33
4.1. Standard way to deploy 33
4.1.1 Gunicorn 34
4.1.2. Reverse proxy as Nginx 34
4.1.3. Deployment summary 36
4.2. OpenStack Cloud deployment 36
5. Final summary 39
Reference list 40
Links 42

INTRODUCTION / BEVEZETÉS

1. Background of the topic

Over the last decade technology of automatic speech recognition (ASR) and Voice-to-Text in general have seen a growth in popularity. It is being implemented and deployed across a wide range of domains, beginning with virtual assistants like Siri by Apple or Alexa by Amazon, finishing by accessibility services for deaf. Starting from the year 2015 the number of articles and literature in ASR increased significantly [1]. According to industry estimates, the global speech recognition market exceeded 7 billion US dollars in 2024 and is projected to reach 8.77 in the year 2025. Expert Bergur Thormundsson from Statista Market Insights forecasts the expected annual growth rate of approximately 18 percent with the total increase of almost 170 percent by the year 2031 compared to 2025 [2]. This is clearly visible in figure 1.1.

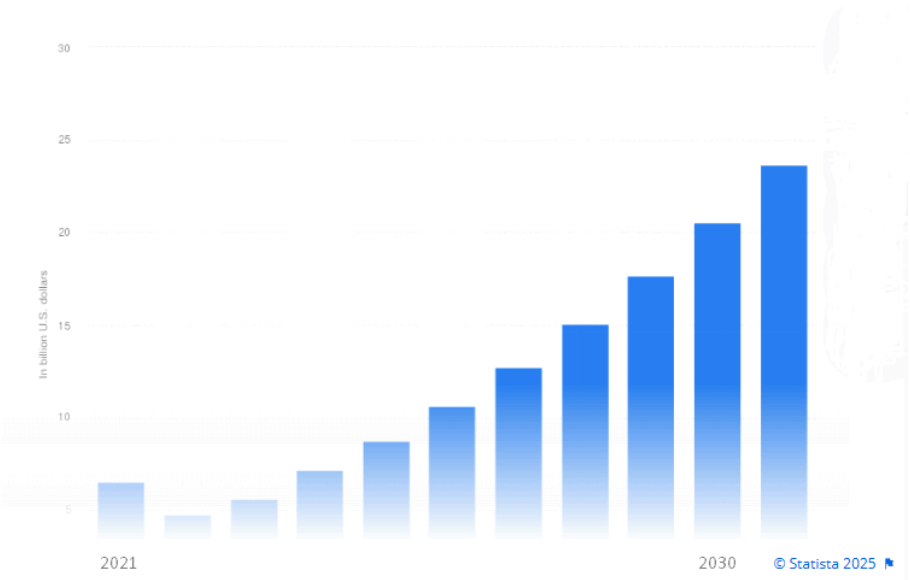


Figure 1.1 [2]

1.1. Motivation

Despite this rapid adoption and wide range of already developed solutions, integration of a high quality Voice-to-Text into a custom application or a pipeline remains challenging:

- **Cost and vendor lock-in.** Cloud based API providers such as Amazon Transcribe or Azure Speech Services have a Pay-Per-Use model, charging per minute of audio and

typically does not allow fully offline on-premise deployment. For small projects such recurring fees and having all your data go through the cloud every iteration can be a huge downside.

- **Integration complexity.** Although providers usually do offer REST (representational state transfer) or gRPC (high-performance remote procedure call framework), implementing them into an existing, say web-application or mobile-app can be labour and time consuming, requiring extra authentication services, network-retries and many other. Sometimes this can cause conflicts with the existing data encryption or network libraries.
- **Privacy and latency.** Sending locally recorded audio files to remote/cloud servers can induce delays in real-time applications, and certainly may violate privacy regulations if sensitive data is being routed through public clouds.
- **Limited languages and accents support.** Premade models may underperform on some niche dialects or non-popular accents. Consequently, custom finetuning on some local datasets may be needed.

On account of the above, a growing need in self-hosted, easily integrable and lightweight solutions for Voice-to-Text can be seen. Particularly, solutions which can be implemented and tailored to a specific dialect and implemented into an application with minimal overhead.

1.2. Existing solutions overview

In order to find the best starting direction we should briefly go through the number of the most popular solutions and finalize their advantages and disadvantages.

Firstly let's talk about cloud based ones which in theory should be the most powerful, because of the larger datasets of the global companies offering them.

- **Google Cloud Speech-to-Text [3].** Supports over 125 languages, providing punctuation. Offers synchronous and streaming endpoints. On the other hand, it has pay-as-you-go pricing and requires uploading audio to the endpoints via API keys, which also need to be managed, all above is not trivial for small projects.
- **Amazon Transcribe [4].** Can be easily and deeply integrated into the AWS cloud ecosystem. However, it does not permit local model finetuning and once again requires constant and stable connection.

- **Azure Speech Services [5].** Provides a SDK for Speech-to-Text and Text-to-Speech with the ability to translate the messages on the go. With billing per audio hour and SDK being relatively heavy it is acceptable for larger projects and companies.

Summarising aforesaid, such solutions are a good fit for larger companies or the projects which bet on a future development.

Now let's move on to open-source solutions, which are said to allow higher flexibility.

- **Kaldi and CMU Sphinx.** Toolkit that allows self-hosting and finetuning. However, it requires setting up pipelines, creation of feature extraction scripts and other.
- **Mozilla DeepSpeech**. Prebuilt python bindings. While being simpler compared to ones mentioned above, default models are quite large and runtime dependencies can be hard to install (in most cases requires GPU support, for instance).
- **wav2letter++ and ESPnet.** Frameworks with C++ backend and TensorFlow models. To be short - not an easy "pip install" integration.

While self-hosted open source solutions exist, most of them require advanced knowledge in speech-recognition, which is a hit if a team wants a fast prototype prior to development. To be said, that team will require a specialist to work on that.

1.3. Research Gap

Bridging the divide between mentioned above "costly and easy" cloud services and "open source and complex" solutions, this paper will provide a complete way to create an application or a script which will use robust solutions for speech-recognition, while using highly integrable and elegant frameworks.

Now all the components of such a system will be stated superficially with further chapters revealing all the nuances and reasoning behind such choices.

- **Finetuning pipeline** based on Whisper (OpenAI) architecture with efficient adaptation to new languages using as few as 5000 audio-text pair samples, even allowing "streaming" as an option to conserve the disk space while using larger datasets.
- **Python/Flask framework web service** having minimal dependencies and allowing ready-to-go integration with any existing web application.
- **Deployment** comparing different options like openstack cloud and standard Gunicorn+Nginx bundle.

2. Model Fine-Tuning

2.1. Why Whisper?

When creating a custom Voice-to-Text project, one of the first and consequential decisions is which pre-trained model to use as a base. Nowadays, OpenAI's Whisper models [6] built its reputation as a leading open source solution offering high transcription accuracy across many languages and proper performance even if audio is decently noisy. Now, let's embark on how it is compared to the rivals, how it is structured and what changes had to be made to use it.

2.1.1. Comparison to the alternatives

According to other researchers Whisper tends to achieve lower word-error rates (WER) than other ASR systems, which is shown in the figure 2.1 which was taken from the K. Kuhn's paper [7]. For English and major European languages even Whisper Tiny and Small report sub 10 percent WER on clean speech benchmarks. The initial tests of Whisper in the previously discussed paper were done on the Large-v2 model, which is far more complex and accurate, but is an overkill for this project.

Vendor	English	LibriSpeech	ESL	German
Amazon	4.4	6.1	7.1	18.0
AssemblyAI	4.5	4.2	5.9	13.2
Deepgram	8.3	12.8	11.4	19.3
Google	20.1	23.6	28.1	18.1
IBM	11.2	13.2	17.3	20.6
Microsoft	4.4	5.9	6.7	10.1
Rev AI	4.4	7.0	6.7	19.2
SpeechText.AI	8.4	11.3	14.1	21.4
Speechmatics	3.3	3.6	4.6	8.0
Tencent	4.6	4.1	7.3	-
Whisper (large-v2)	**2.9**	**3.3**	**3.3**	**5.0**
Average	7.0	8.6	10.2	15.3

Figure 2.1

One of the perks of Whisper is that it was jointly pre-trained on several hundreds of hours of multilingual supervised data. This gives it out of the box support for almost a hundred spoken languages. According to different papers, Whisper's training set deliberately included challenging audios (filled with background noise, phone call recordings and other). Consequently, it should be good at transcribing "real world" audio.

2.1.2. Why Whisper Tiny?

Whisper consists of 5 models, which can be seen in table 2.1.

Table 2.1

Model	Parameters	Speed	Disk Size
Tiny	39 M	fastest	~150mb
Base	74 M	fast	~300mb
Small	244 M	moderate	~760mb
Medium	769 M	slow	~2.4gb
Large	1551 M	slowest	~5.9gb

Project's development environment uses a single consumer-grade GPU: RTX3050 with 6 GB of VRAM. Whisper's tiny 39M parameters even with mixed-precision fit in memory, while even Small with its 244M can exhaust VRAM under batch sizes more than 16 (which were used in our case). A 1000 step finetuning on 5000 samples usually completes in around 2 hours. Running finetuning with the same set of parameters using Whisper Small would take almost triple the time. Moreover, the primary goal of this project is to showcase that even light finetuning can result in a functional model giving immediate iteration cycles: finetuning of hyperparameters, building data pipelines, integration with the Flask app.

2.1.3. Architecture explained

This paper will not dive deep into the architecture of Whisper, but it will highlight and explain important aspects of it. According to the official web resource of OpenAI combined

with the publication of David Cochard on Medium [8] Whisper uses a standard encoder-decoder transformer. Tiny has 6 encoder layers and 6 decoder layers, each with 384 hidden units and 6 attention heads, resulting in around 39 million trainable parameters. The general architecture can be seen in figure 2.2.

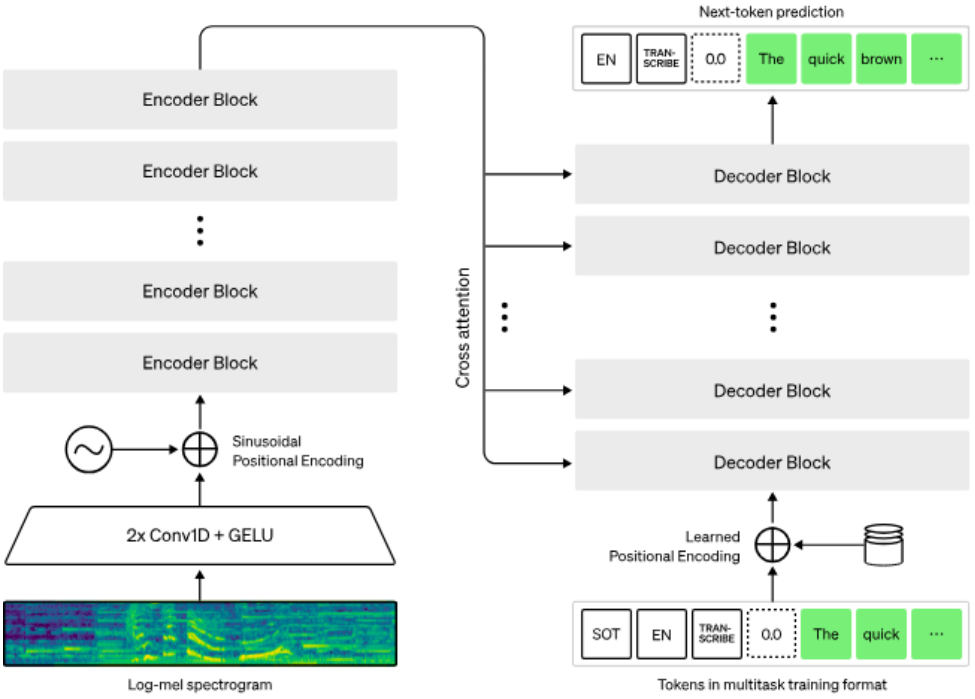


Figure 2.2 [6]

Execution flow:

- Preparation: Audio files are handled in 16 kHz format and are converted to a Mel spectrogram.
- Here the language identification starts: Once every 30 seconds the encoder runs, converting the spectrogram to a feature vector. The decoder outputs the probability to each one of the tokens from the latent representation. Language is being determined based on token ID corresponding to the output of the decoder.
- Transcribing: feature vectors from the encoder are passed to the decoder, which outputs sequence of tokens.
- Timestamps: timestamp token is checked in decoder output to see how much time of text was transcribed.

- Do that in iterations.

To sum up this chapter, Whisper Tiny is hitting the ideal balance for this project, delivering solid ASR performance, multilingual capabilities and low resource requirements. It allowed us to finish our development iterations faster, proving the concept before scaling up.

2.2. Why Huggingface interface?

Whisper provides two different scripts to run the finetuning pipeline: on custom dataset or huggingface dataset. For this project the latter option was chosen. It could preserve local disk space, avoid the need to manually manage tar balls and it easily integrates into Python scripts with the future ability to save and load custom tuned models using one line.

2.2.1. Huggingface package

At its core, Huggingface (HF) [9] is a simple Python package which can be installed using PIP and works closely with the Datasets package. After obtaining the HF token user can simply log in in the IDE's terminal and call load_dataset function from the Datasets library.
It can be as simple as:

```
from datasets import load_dataset
ds = load_dataset("mozilla-foundation/common_voice_16_1", "ru", split="train")
```

Overall you have:

- A uniform API for different audio collections (like common voice, voxceleb), vision datasets (imagenet) and text classifications (IMDb).
- Auto download and verification, hence no need to hardcode URLs for datasets or unpack them with the "continue download" function active, allowing to resume downloading after "connection lost".

2.2.2. Streaming mode and Caching

One of the innovative functions that were used in this project is streaming mode. This is a flag (`"streaming=True"`) that allows you not to download the complete dataset upfront, but rather fetch one record at a time over HTTP, saving the disk space needed. It is useful, if you are slicing the dataset in order not to use all the samples from it. This flag is turned on in huggingface for both common_voice 11 and 16, which were used in the project and by default they have only splits: Train, Validation, Test, Other and Invalidated. And you can slide the one you need. Such streaming datasets never fully materialize in the memory, consequently even on a test server with limited storage the whole pipeline can be run end-to-end.

Since the project is being developed on a private machine the network connections can be unstable. Huggingface datasets provides some useful features ro address this problem:

- Local caching: once any file (or its part) is successfully downloaded it moves to a global .cache directory, hence if your script will be abruptly stopped by some error, repeated runs will not redownload the same files.
- Automatic retry: HTTP or S3 errors are retried by default without extra parameters needed to pass.

2.2.3. Mozilla Common Voice

For the finetuning experiments, Mozilla Common Voice [10] was selected. It is an open source, crowd sourced speech dataset with many advantages:

- Open license: all the recordings and transcripts are released into the public domain, which means can be legally used for academic or demonstration purposes.
- Rich metadata: each audio sample includes description about its quality, speaker demographics and dialect/accent labels.
- Multilingual coverage: it has different releases for almost 100 languages, including the ones used in the project - Hungarian, Russian, English.

To sum up, this way we obtained a well-maintained and robust ecosystem for data gaining allowing simple parameters and tags swaps for changing dataset versions, loading types and other. Moreover, most finetuning scripts allow usage of huggingface+datasets bind out of the box with minimal changes needed.

2.3. Structure of finetune script

Whisper provides two scripts for finetuning: finetune on custom and huggingface datasets and in this project the latter one was used. It is pretty basic and acts as a base script that can be modified. To begin with, the standard structure of it shall be briefly explained. Script consists of several main blocks needed for: argument parsing; loading the model and processor; data preparation; training loop.

2.3.1. Argument parsing

At the top of the script a `parse_args()` function is located. It is built on Python's "argparse" library and acts as a hyperparameter holder and receiver. Below the example of setting for the model selection can be seen:

```
   parser.add_argument(
        '--model_name',
        type=str,
        required=False,
        default='openai/whisper-small',
        help='Huggingface model name to fine-tune. Eg: openai/whisper-small')
```

This makes each parameter configurable from the command line. For instance, you can specify the defaults, in this case without explicitly setting up the model from the CLI, the finetuning will run on a Whisper's Small model. Also, you can override the defaults without touching the code, using the same example again you can add "`--model_name openai/whisper-tiny`" tag to use Whisper Tiny.

By centralizing all hyperparameters and defaults for them what and how a user wants to run the script.

2.3.2. Model loading

Once the script gets the arguments from the command line the next step is to load the model, which can be done as simple as that:

```
processor = WhisperProcessor.from_pretrained(
   args.model_name,
   language=args.language,
   task="transcribe")
model = WhisperForConditionalGeneration.from_pretrained(args.model_name)
```

Here "`WhisperProcessor`" bundles together audio feature extractor (spectrograms) and tokenizer (text into token IDs). And "`from_pretrained`" fetches the weights, configuration and vocabulary from the argument parser (in this case it checks what model to load and what language the user has chosen).

2.3.3. Data preparation

Here the pipeline consists of two main functions:

- "`load_all_datasets()`": this function was explained in chapters 2.2.1 and 2.2.2 the only thing to add here is that it also receives the set of chosen arguments (this way the function knows which dataset and its split to load).
- "`prepare_dataset()`": it takes a single record of raw audio with its text and do several steps - resamples the waveform, computes the log-Mel spectrogram and tokenizes the transcript to produce labels.

After all these steps slicing (usually with shuffling) and filtering (removes overly short and long recordings, ensuring a uniform batch performance). Both mapping and filtering steps write their own outputs to a disk cache, hence if you are experiencing train fail it will not redo already finished steps.

2.3.4. Training loop

Finally, a "`Seq2SeqTrainingArguments`" object is being constructed. It collects all the defaults and user set parameters to specify learning rate, batch sizes, output directories and other arguments. This object is passed to "`Seq2SeqTrainer`" which specifies the model, training arguments, callbacks for checkpoints and datasets to train and evaluate on. A single call to "`.train()`" afterwards will run the whole finetuning and save the final model into the set up directory.

2.4. Customizations to the finetune script

Whisper base training script provides a solid baseline, but for this project a few enhancements were introduced. Ones to save time on time consuming actions, ones to gain a bit more performance out of available for the project hardware. Below the main customizations will be described.

2.4.1. Encapsulation and changes of data preparation

As it usually happens, it is hard to build all the needed dependencies correctly, which can result in some functionality being unavailable from one machine to another, one of the missing abilities was running mapping in parallel. Hence, after all the datasets were loaded it took approximately 40 minutes to map the records. Encapsulation of the whole script helped to solve this issue. Now you can parse the "`--num_proc`" parameter, which controls the number of parallel jobs to run, which saves almost 90 percent of the time originally needed to complete the data preparation step.

The other crucial change was made to a loading of datasets itself. Now two options are available. You can either run the downloaded in the streaming mode (which pros were discussed in chapter 2.2.2) with the ability to slice the dataset prior to its full downloading, or you can decide to spend your time obtaining the whole dataset and only after to a slicing and shuffling. Below you can see the part of the code which is responsible for streaming mode:

```
for i, ds in enumerate(args.train_datasets):
   train_split = args.train_dataset_splits[i]
   raw = load_dataset( ds, args.train_dataset_configs[i],
        split=train_split, download_config=dlc,
        streaming=True, trust_remote_code=True, )
   dataset = raw
...
```

2.4.2. GPU training

Traditionally users can start the training via a plain python call in the terminal. But once again the complications of training using a personal machine appeared and GPU was not

being recognised and the whole training was taking close to 6 hours with one iteration being around 30 seconds. The solution was to use Accelerate [11] toolkit. This toolkit auto-detects the environment and wraps the processes accordingly, whether you have no GPU, a single GPU etcetera - no need to manually set "`cuda_visible_devices`" or "`torch.distribute.launch`". Also, usage of flags for optimisers can be used again. For instance, BitsAndBytes' 8-bit optimizer, which allows a user to squeeze more batch size into GPUs.

Summing up all above, now there is a script and bash command combination, which can be used on different machines, servers and others without a need to change a single line of code or a flag.

2.5. Dataset composition and training

For finetuning experiments three Common Voice releases were selected (on per chosen language). Fixed-sized small subsets of them were used for training, so each run would be completed in around 2-3 hours using a single RTX 3050 GPU. Summary can be seen in table 2.2. To note, shuffling was always applied using "`.shuffle(seed=...)`" to randomize selection.

Table 2.2

Language	**Common Voice release**	**Full train split**	**Subset (train/eval)**
Hungarian	16.0	36.5k samples	5000/1000
English	11.0	112k samples	5000/1000
Russian	16.0	26.3k samples	5000/1000

Example of training command for english language can be seen below:

```
accelerate launch .\train\fine-tune_on_hf_dataset.py `
  --model_name               openai/whisper-tiny `
  --language                 en `
  --sampling_rate            16000 `
  --train_strategy           steps `
  --learning_rate            1e-5 `
  --warmup                   100 `
  --train_batchsize          32 `
  --eval_batchsize           16 `
  --num_steps                1000 `
  --output_dir               ft_quick_tiny_en `
  --train_datasets           mozilla-foundation/common_voice_11_0 `
  --train_dataset_configs    en `
  --train_dataset_splits     train `
  --train_dataset_text_columns sentence `
  --eval_datasets            mozilla-foundation/common_voice_11_0 `
  --eval_dataset_configs     en `
  --eval_dataset_splits      validation `
  --eval_dataset_text_columns sentence
```

Some of the key parameters should be explained:

- **train_batchzise:** A batch size of 32 is the highest value that can be used on the presented GPU taking into account the vRAM. Larger batches risk out of memory error.
- **eval_batchsize:** Size for evaluation was halved to reduce peak memory during validation.
- **warmup:** The value was set to 100, meaning during the first 100 iterations (steps) the learning rate linearly grows from zero to its peak (which was set to 1e-5).
- **train_strategy** and **num_steps:** Instead of choosing batch strategy (which on 5000 subset would result in only a few batches per epoch), the steps strategy was chosen. It allows us to specify the number of optimization steps, which was set to 1000. At 32 samples per iteration it provides fast and sufficient exposure for finetuning (32000 examples - approximately 6 passes over chosen subset).

Same parameters were used for all three languages with the exception of different versions of the dataset and language flags.

2.6. Evaluation of the English model

In order to properly evaluate the results of training a custom script was created. It can be seen in the official project's repository, link to it can be found in the "Links" section of this paper. It calculates Word Error Rate on a small subset of 1000 samples, providing comprehensive metrics and even outputs the worst in terms of WER transcriptions. WER values can be seen in table 2.3.

Table 2.3

Metric	Value
Overall WER	43.23%
Average per-sample WER	43.59%
Median per-sample WER	33.33%
Maximum per-sample WER	1042.86%

A median WER of 33.33 percent, while not being the best, for sure, indicates that one third of the words are recognised incorrectly. Given that the model was trained only on 5000 samples, it can be considered as a solid result. The average WER is pulled up by the number of extreme transcriptions, where the model got stuck into some loop. The 5 worst examples are shown in the table 2.4.

Fails mostly occur on long or complex sentences; rare nouns and abbreviations; and sentences with the unusual punctuation. Only around 5% got WER values higher than 100%. Short, clearly said sentences (everyday speech) tend to have a much lower WER.

Table 2.3

Reference	Result	WER
A decade later, the organization\'s "Newsletter" became a magazine, the "Chalcedon Report"	A decade later, the organisation is now now now now now now now now (“now” - repeatedly)	1042.86%
He attended Porter-Gaud High School.	Amaru Hauru Hauru Hauru Hauru Hauru Hauru Hauru (“Hauru” - repeatedly)	903.46%
Written as a travelogue, Yendred's journey through the West takes him through three cities.	During the day, the day, the day, the day, the day, the day, the day (“the day,” - repeatedly)	781.91%
Your appointment has been changed.	You're at one minute as we teach.	140.00%
I've got indigestion.	Ive got in the gesture.	133.33%

These results show that even Whisper Tiny only trained on a small subset can achieve acceptable results, which definitely proves the concept this paper wanted to achieve. As for future milestones: increasing size of a dataset; changing to a larger model Whisper offer, are suggested. Only these two modifications, would possibly lower the WER to around 10-15 percent.

3. Web-application

In this chapter the design and architecture of the speech-to-text web-application will be described. To begin with, a high-level overview of the chosen stack will be presented (figure 3.1). (All the scripts are available for viewing in the project's repository accessible via link {1}.)

Backend - Flask API: Python based REST endpoints for transcription and authentication, combined with main pages; Model loading and audio preparation inside the auxiliary functions set; close integration with MongoDB as a storage for caching transcripts and storing user credentials

Frontend - UI: Web pages developed using a combination of HTML/CSS/JS; UI consists of several pages for user credential input, administration page and a main page for transcribing.

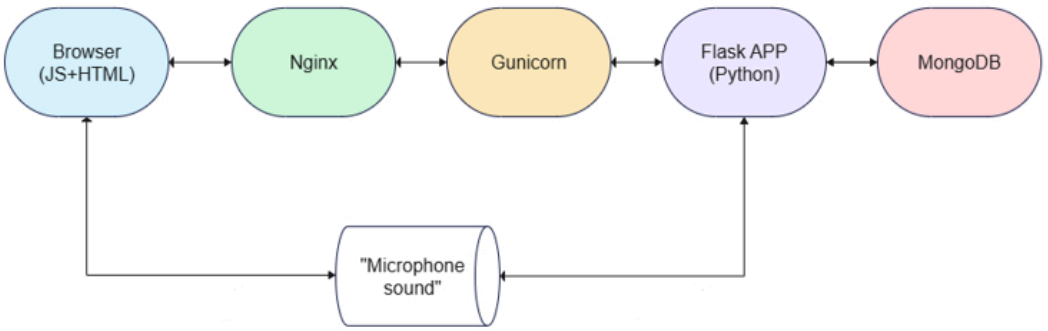


Figure 3.1

3.1. Why Flask?

Flask is a Python based lightweight framework, which is especially good in integration with machine learning and data science libraries. Here is the short list of reasons it was chosen:

- **Core:** Flask provides solid essentials for routing, templating and request/response handling, keeping the footprint of the application small and elegant.

- **Python:** Flask can be integrated into a python application with a single liner, as easy as to import other heavy-duty libraries needed for audio preparation and (Librosa, Soundfile) and Whisper model loading (Torch, Transformers).
- **Blueprints**: Flask blueprints allow modularity, which simplifies the code and creates the environment for the future growth of the application; for instance, project authentication was structured as a blueprint which can be called via simple "`app.register_blueprint(auth)`".
- **Sessioning:** Flask's session object enables signed-cookie based sessions, allowing to store minimal needed info, such as used ID and role, in the sessions, separating them from credentials stored in the database.
- **WSGI:** Flask applications are compatible with Web Server Gateway Interface - they work with any WSGI server like Gunicorn and reverse proxy like Nginx; such compatibility and portability ensures our deployment will be successful either in OpenStack or plainly on the server.

3.2. Backend structure

Now let's dive deeper into the project's backend. Flask-based backend consists of one main module, two secondary models and a set of "helper" functions.

Let's start with the smallest of the bunch - "db.py": it is a simple database connector utilizing the PyMongo library and which is loaded into the main module via standard python import. It loads two collections: cached transcriptions and users' ID and credentials if needed. Decision to make a separate connector configuration appeared in order to be able to simply adapt it if any future changes happen.

Next module is a "auth.py": it is a complete set of endpoints and logic for user login, registration and session handling. It utilizes the Werkzeug (is a comprehensive WSGI web application library) for password hashing, so no raw credentials data is being pushed to the database. Endpoints can be seen in the Swagger specification in the figure 3.2. Now let's go through each of them (table 3.1):

Table 3.1

Methods	Endpoint	Description
GET	/register	loads registration form
POST	/register	collects user credentials and form a database request to create a new user; if successful - renders the login form
GET	/login	loads login form
POST	/login	collects user credentials and forms a database request to check if they are valid; if successful - redirects to the main page and create a new session for authenticated user
GET	/logout	log out current user, clearing the current session

Now let's see the overview of the final module - "app.py": it is the application entry point, which imports and registers the "auth" blueprints. It also defines all other non-authentication routes. There are three hepler functions defined in it. Description of all them is available for preview in table 3.2.

Table 3.2

Name	Input variables	Output variables	Description
get_transcripts_collection()	none	db["transcripts"]	Simple database call utilizing database connector to load cached transcripts for a specific user session; implemented a logic - “fast to fail”, so gives out an unavailable response if the database is down.
transcribe_audio_array(..)	selected, audio, sampling_rate	transcription = proc.tokenizer.batch_decode(..)	Collects the user microphone recording and a current active model to run a transcribe on it; checks if the audio has the correct sampling rate; also checks the “device” to see what mode to run (cpu or gpu)
model_loader	model_paths	Model, Processor	Checks the current flag for the language model user wants to use and loads it via global variable consisting of key and path; also checks for cuda available devices, if not successful - runs in cpu mode; encapsulated in the main part of the app, hence the models are always preloaded.

Figure 3.2 illustrates the endpoints, methods and a short description of each of them for the main module.

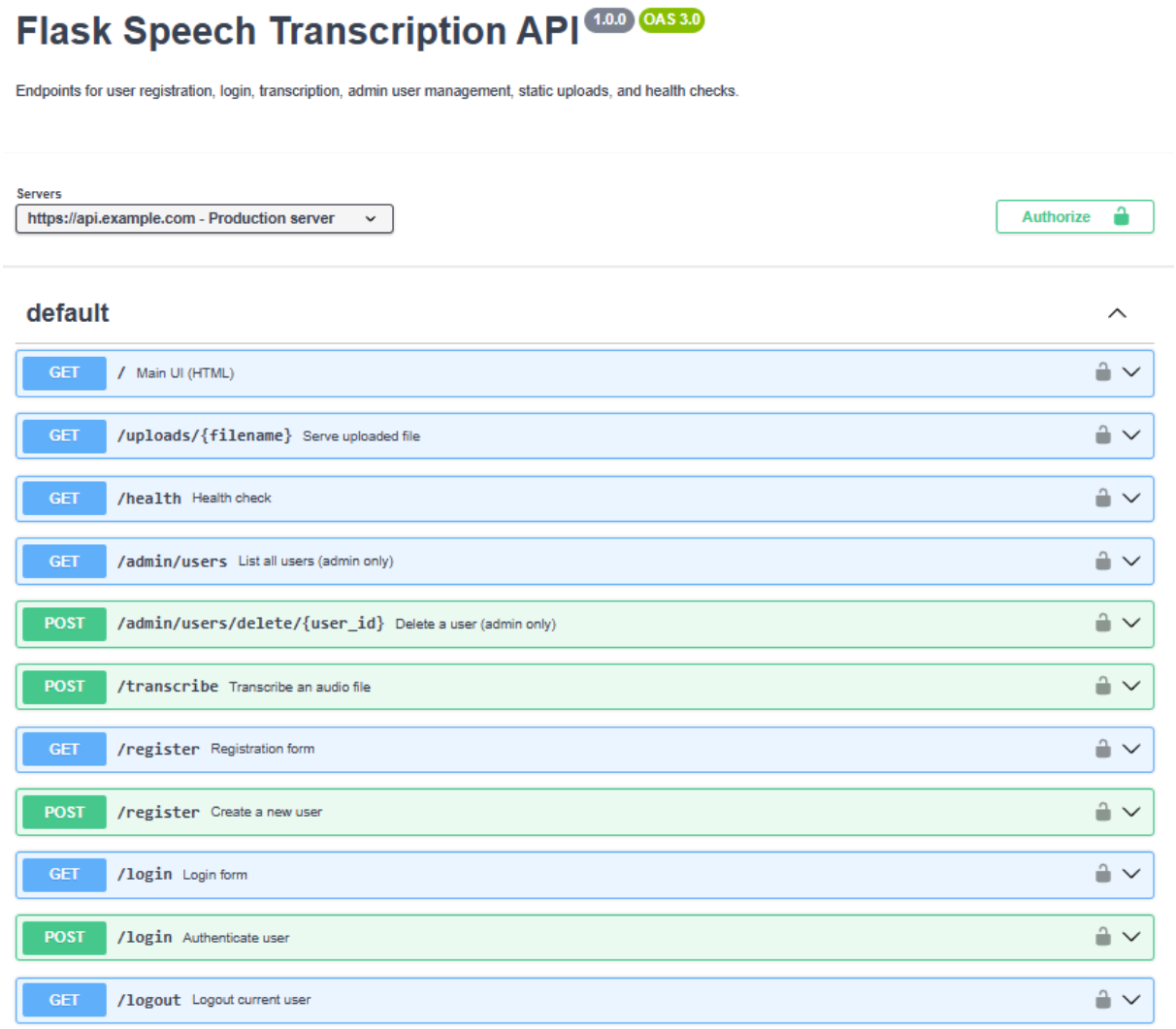


Figure 3.2

For all the API calls the HTTP response status codes are defined. That way a developer and a user (in some cases) is able to check and see if everything works as intended and not a single part of the data pipeline or request if failing. Moreover, it is possible to check the validity of created endpoints prior deployment via specialized tools like postman, whose documentation also provides useful guides on how to set the response codes to development standards. All the available status codes will be shown (table 3.3):

Table 3.3

Code	When it appears?
200	When the request was successful (sometimes it is 201 for post requests - successful creation of a record)
304	When app successfully loads static scripts like “scripts.js”
400	When somehow the model is not selected or the audio recording went missing (or both)
401	When a unauthenticated user tries to access the “transcribe: page
403	When a user is authenticated but authorized to check the page for user administration
500	When a internal server error happens

3.3. MongoDB structure

As for project storage, MongoDB was chosen. The reasoning behind this decision is quite simple: the scaling is projected to be only vertical (when resources are added to the existing system without changing its configuration). In addition, recent studies showed that execution times of operations such as search and insertion are up to 50 percent faster in MongoDB compared to regular relational databases (the tests were comparing MySQL and MongoDB on 10000 records, which is the estimated number of accumulated records in several months after deployment) [12]. Also, using non relational databases does allow for faster application building, diverse data storage and easier management at a large scale (in perspective). Moreover, non relational databases with their key-value pairs (which can be considered as a drawback for some systems, but in some - can be a plus) are integrable into the cloud bucket storage logic, hence all the details of a user (for instance) can be obtained by a single request for a single record. Figures 3.3 and 3.4 show the examples of two collections of this project’s database.



Figure 3.3

First one is a “transcription” record type, it is developed to hold all needed information to track the history of transcribe action for a set of users.

- “**_id**” and “**user_id**”: represents the identification number for a record itself, while user_id maps the records to the record of a user in the second collection, allowing for a fast query search among all of the transcriptions.
- “**model**”: shows the model used for the transcription and currently there just for a history, it does not appear in the “past transcripts” table on the page.
- “**text**” and ”**timestamp**”: represents the most vital data for a user, which does appear in the table, when the user loads his past transcriptions.
-

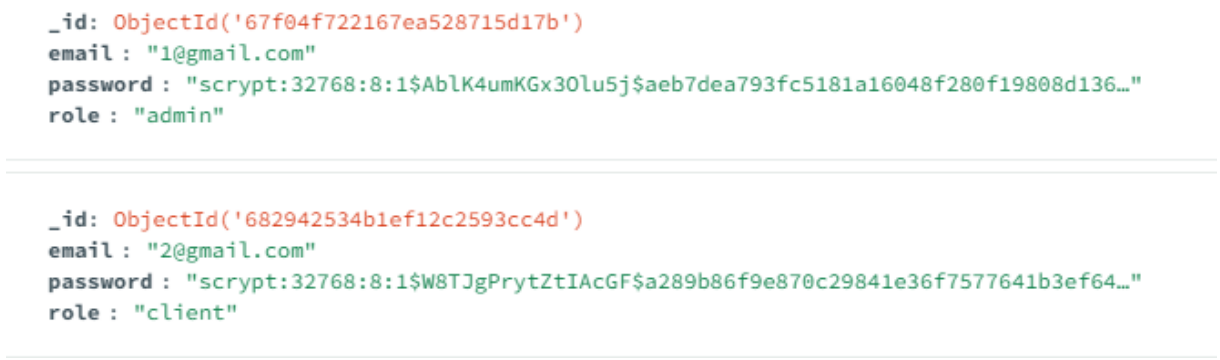


Figure 3.4

Second one is used for storing the user’s personal data. The important thing here is that the unencrypted passwords are never stored here. The library used for this action was

described in section 3.2. Besides, the role data is also stored here. It is used by a sessioning logic to display and allow admins access the pages hidden to a regular user.

3.4. Frontend Structure

Project's client side code lives under two directories shown on figure 3.5. The right folder contains 4 HTMP templates for loading a web page structure. Each HTML file imports static files in the "`<head>`". Style (style sheet language) defines the visual appearance of the pages: dark theme, button styles and responsive grid for two columns on the main page (recorder and transcripts history). Also, the model selector switch is implemented here. It is a simple "`<select id= .. > <option value = ..> </select>`" structure: user clicks on a model selector field and chooses a model from a dropout menu. Session check for a user role also implemented here via "`{% if session.role == 'admin' %} ..`" to show a button for accessing the part of the app used for administration. "`<audio class="dark-player" controls></audio>`" element is hidden until the user records an audio for the transcription.

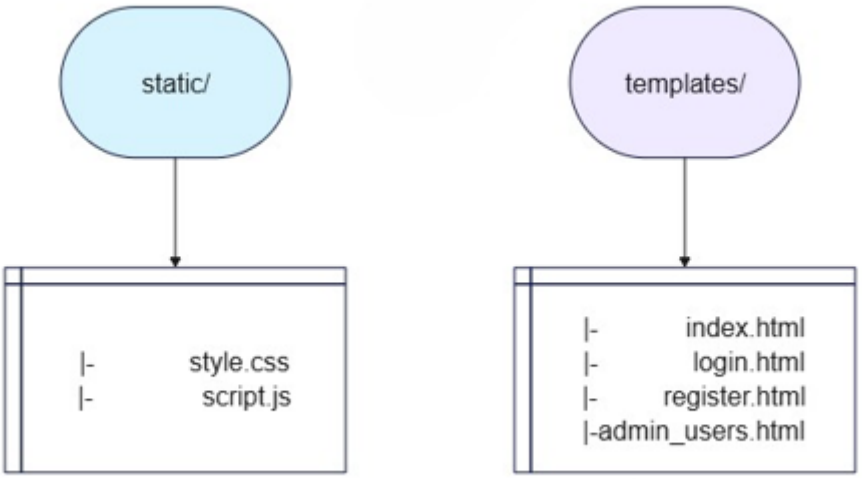


Figure 3.5

For the button presses, redirections and the entire logic JavaScript was used. It works as simply as: script grabs UI elements and prepares media recorder for raw audio input; then event listeners monitor position of a record button. When it is pressed the first time it swaps the style of it to a "stop" button and records an audio input. After the second press, script

stops the recording and updates the “current transcription” label with a new transcription and sends the older one to a history block, which is being reloaded as well.

An elegant solution here is never writing the recording file to the disk storage. With a relatively simple code:

```
    const audioBlob = new Blob(audioChunks, { type: 'audio/webm' });
    const formData = new FormData();
    formData.append("audio", audioBlob, "recording.webm");
    ...
    const url = URL.createObjectURL(audioBlob);
    audioPlayer.src = url;
    audioPlayer.controls = true;
    audioWrapper.style.display = 'block';
```

Basically, the recording lives in a form of raw bytes in JavaScript’s blob and gets then passed to the audio player via a temporary object url. After the recording is ready, parallel pipeline activates fetching it to transcribe endpoint, and as said before to a UI media player. This way two problems are getting eliminated at once: UI gets the audio player ready instantly, no matter when (or even “if”) the transcription comes back; and we are neither cluttering up neither the memory of a server (if recording are to be sent there), nor overloading the user machines RAM (since we are discarding the previous audio files as soon as the new one starts recording).

Some things to add here:

- audio is passed in chunks of around 200 ms, so as the browser accumulates such chunks it triggers the event listener for “dataavailable” and simply moves it to the temporary memory.
- There is a total limit of a recording set to 30 seconds, hence after 30 seconds even if the user never clicks the stop button, the process will be stopped automatically. There are several reasons behind the decision to cap the maximum length of the audio. Firstly, as described in chapter 2.1.3. the encoder runs on every 30 seconds split of audio outputting the feature vector, that means we will have only one iteration of that process. In addition, as it is easy to figure out, a model trained on only 5000 samples is

not the best in transcribing long and complex sentences and tend to enter the loop in such cases, which is exactly what needs to be eliminated at this stage.

3.5. UI overview

Now is a good time to talk about the actual UI of the application. Since users nowadays tend to use mobile devices which in most cases have an OLED display (OLED technology started to enter the PC/Laptop monitor industry a few years ago, as well) a decision was made to use a dark theme across the whole application. Figure 3.6 and 3.7 illustrates the login and register pages. Both of them have two text fields for user credentials, and a few buttons to press. If a user tries to create a user with existing email in the database or enters invalid data - the backend will call a function to rerender existing page and display the red warning message:

```
if not user or not check_password_hash(user["password"], password):
render_template("login.html", error="Invalid credentials")
```

or

```
if users.find_one({"email": email}):
render_template("register.html", error="User already exists / Invalid credentials")
```

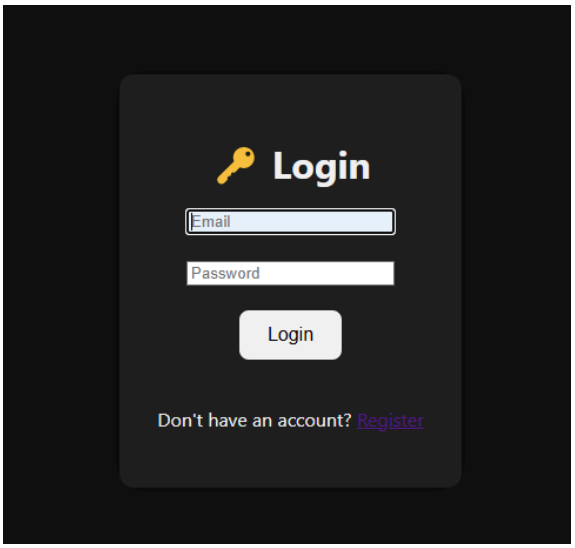


Figure 3.6

Also, if you manage to correctly register a new user you will be automatically redirected to a login page.

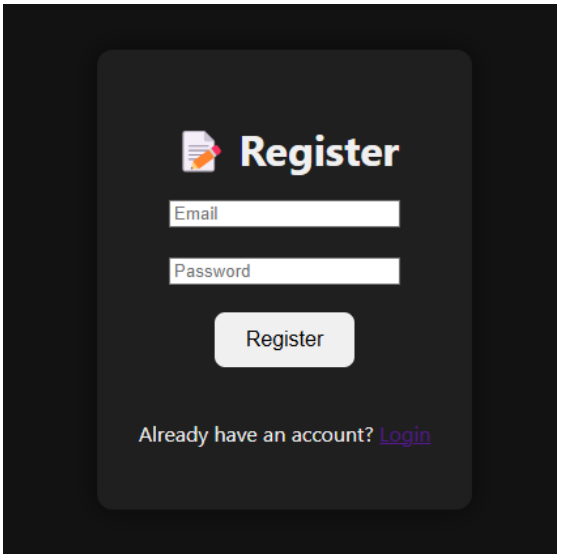


Figure 3.7

Next part of the UI is the main page, which is shown in the figure 3.8. It has three main parts of the interface:

- top row;
- bottom row, left column;
- bottom row, right column.

All that's done by a responsive grid which adapts to the resolution, aspect ratio and the number of the parameter inside the grid elements itself.

First is the top row which is displaying the currently logged in user, his role and a logout button as defaults. The button “Manage Users” is visible if and only if the session check outputs that the user’s role is “Admin”.

In the middle left part of the screen the area for current transcription is located. It has a dropout menu to select a desired model (three choices: Hungarian, Russian, English) and a button to record, which turns read after you click it (to mimic any audio recording software). Below is a text box used to display either nothing (if no actions were done) or to display the current transcription. On the right the area for the cached transcriptions is located. Up top a media player sits. It is capable of playing the last audio a user has recorded. After a user starts a new recording and clicks “stop” the previous “current” transcription moves to the history table, and a new one appears on the left.

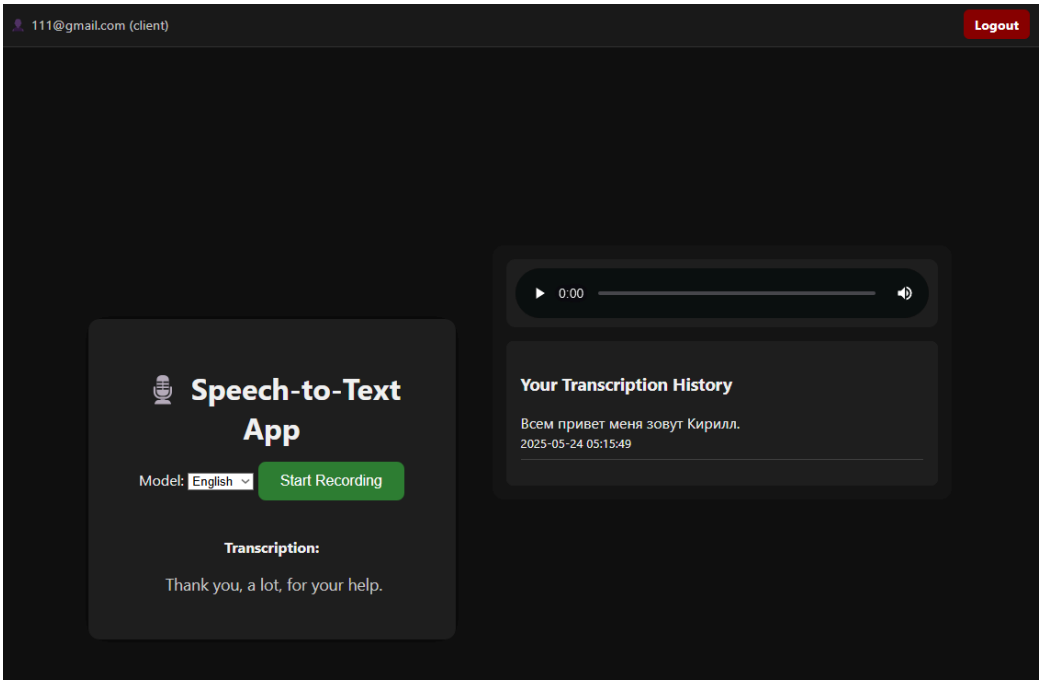


Figure 3.8

Figure 3.9 illustrates the page for administration. As said before, it is only available for the users with an admin role. You have an orange button up top to be able to get redirected back to the main page. In the middle a table, which displays all existing users, is located and admins can check the credentials (apart from the sensible data - passwords) and delete the user if needed.

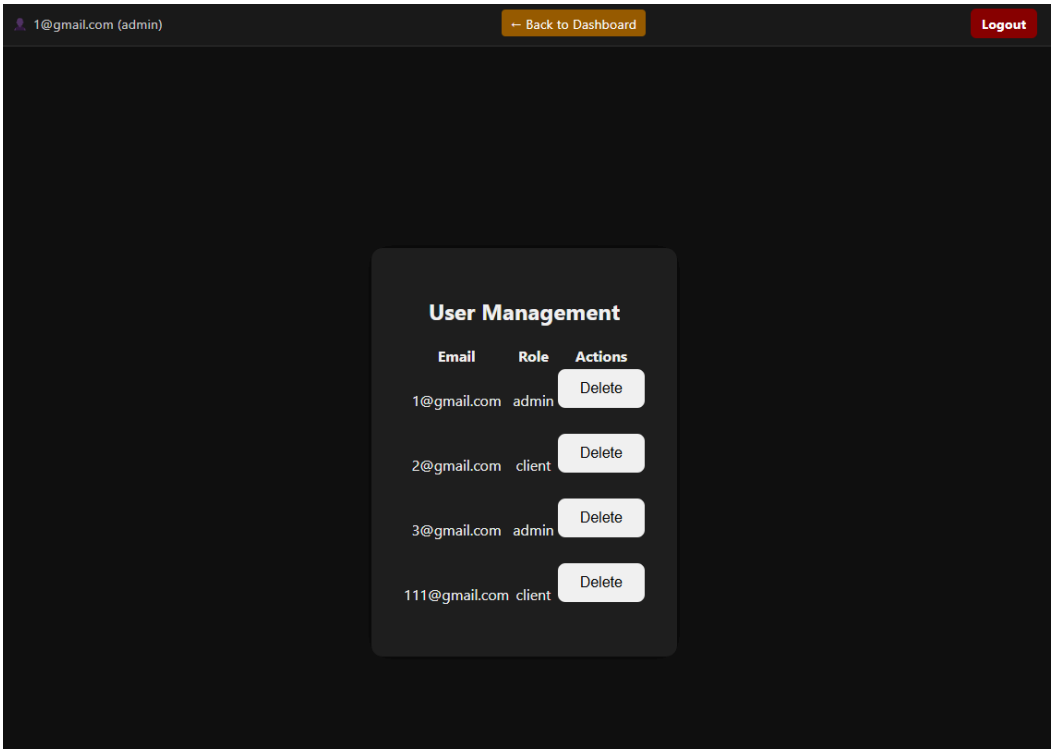


Figure 3.9

4. Deployment

Before beginning this chapter, it would be necessary to talk about version control for the project. All project's code - scripts, templates, static files and configurations - is uploaded to Git repository. For efficient version control the branching strategy applied:

- Feature branch (say, dev_v2.3) is created for a new feature or a bug fix
- Local commits collect all the changes made to any files (apart from specified in gitignore)
- When developer is ready to push his changes he sends every change to a local dev branch
- A pull request opens and starts the process for code review and merge approvals
- One pull request is approved, dev_v2.3 is merged into dev
- Periodically or upon a developer request dev is merged to main

The application is never being developed on a production server, every change and feature is tested locally and then pushed to git. On the server there is a chosen directory where a github repository is initialized. Hence with just a few lines of bash script it is possible to pull the changes or new files from the remote repository.

The pipeline for the model files is quite different, since even finetuned Whisper Tiny's size is close to 1 GB. There is a size cap in Git for regular users that equals to a 100 MB. Consequently, there are several steps to transfer them to a server:

- Each of three models are archived as a tar.gz on the local machine
- After they are transferred via linux secure copy bash command

("`scp tinyru.tar.gz user@0.0.0.0 /local/path /external/path`")

- Upon arrival they are unpacked and ready to be used

4.1. Standard way to deploy

In order to test and polish the application standard deployment was chosen. Hence in this chapter the information of a gunicorn+nginx combination will be presented and later (in the next chapter) compared with the cloud deployment.

4.1.1 Gunicorn

A Gunicorn (Green unicorn) [13] was chosen as a Python WSGI HTTP server. It sits between the raw flask application and the “outside” web.

- **Stability:** workers can handle requests in parallel, if one crashes, it will be respawned
- **Flexible configuration:** number of workers, timeouts, logging - all of that and more is configurable

    - **Integration with Flask:** out of the box compatibility with Flask applications, pointing Gunicorn on the app object will deploy it

We used such configuration to launch the app:

```
gunicorn --workers=3
--threads=2
--worker-class=gthread
--bind 127.0.0.1:8000
--log-level=info "app:app"
```

Let’s go through the parameters:

- **workers:** starts desired number of processes, each of them is able to serve requests
- **threads** and **worker-class:** specifies how many threads can a single worker handle
- **bind:** local host with a port 8000 was chosen here - gunicorn listens on 127.0.0.1:8000 (we do not expose the Gunicorn directly to the web)
- **“app:app”:** just an entry point and a application file to import

4.1.2. Reverse proxy as Nginx

Although Gunicorn is perfect at running python based applications, it is not as optimized for loading static files, managing large number client connections etcetera. Hence, Nginx [14] was chosen to fill this gap. Here are some cases where it can be used:

- Encryption of HTTPS traffic from Gunicorn
- Collecting all incoming requests from the internet and forwarding it to Gunicorn’s port over HTTP, isolation it from the web (reverse proxy)
- Direct serving the CSS and JS scripts (as well as other static assets)

The thing here that needs to be viewed is a configuration file for the nginx:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name 10.3.7.27;
    return 301 https://$host$request_uri;
}
```

This block constantly listens on the specified port for HTTP and redirects all traffic to HTTPS connection.

```
server {
    listen 443 ssl http2 default_server;
    listen [::]:443 ssl http2 default_server;
    server_name 10.3.7.27;

    #self-signed cert
    ssl_certificate     /etc/nginx/ssl/speechapp.crt;
    ssl_certificate_key /etc/nginx/ssl/speechapp.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         HIGH:!aNULL:!MD5;...
```

This block constantly listens on the specified HTTPS port and handles self-signed certificates (this needed for the web browser to be able to handle secure connection and the web application to ask for a microphone permission, without certificates and HTTPS any browser will write off the web application as non secure and only load it with basic unchangeable permissions; it will still warn the user about “self-signed” certificates being not secure, but user will be able to choose if he wants to trust a website)

```
    location ^~ /static/ {
        alias ~/speech-app/thesis/speech-to-text-app/static/;
        expires 30d;
        add_header Cache-Control "public, must-revalidate, proxy-revalidate";
        try_files $uri =404;
    }
    location ~ \.css$ {
        alias ~/speech-app/thesis/speech-to-text-app/static/;
        expires max;
    }
    location ~ \.js$ {
        alias ~/speech-app/thesis/speech-to-text-app/static/;
        expires max;
    }
```

"`Location static, .css, .js`" uses alias to map directories' paths to a static memory. Also it creates a response with a 404 HTTP response code, if any of the static files are not accessible.

```
   location / {
        proxy_pass          http://127.0.0.1:8000;
        proxy_set_header    Host                $host;
        proxy_set_header    X-Real-IP           $remote_addr;
        proxy_set_header    X-Forwarded-For     $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto   $scheme;
   }
```

This block constantly redirects web requests to the port on which Gunicorn is running, binding it with the web users.

4.1.3. Deployment summary

To sum up, we have a standard and robust combination of tools which balances between being simple and secure (such pattern is widely used in Python based web service production):

- Maintainability with a GitHub allowing for an easy code updates
- Security with all the requests serving via HTTPS
- Reliability and performance with Gunicorn restarting workers and Nginx efficient static file handling

4.2. OpenStack Cloud deployment

Cloud computing nowadays is getting more and more companies. Large corporations moving their own hosted processes, applications, databases and other to a public cloud providers. But with pay-as-you-go strategies and security risks small projects tend to continue using standard architecture for their project. Here OpenStackCloud comes up with its branches. Generally speaking: fully customizable open source cloud components, which can be scaled as the developer wants. One of the sub projects of OpenStack is Microstack. Which can be extremely useful for project's as described in this paper.

Microstack is an open source project that allows for an easy installation of OpenStack on a single machine [15]. It is delivered by Canonical's snap packaging system. You only need a single line bash command to install a fully integrated toolkit for virtualization, networking and storage. Which is much more convenient for small and single machine projects, since there is no need to install multiple components and bundle them together to make them work. With it you can rapidly set up Virtual Machine, networks and volumes to emulate the production environment for your application.

Core components of MicroStack:

- **Keystone:** Identity service for managing users and user roles (after configuring an admin user, tool automatically sends the auth-token for the dashboard)
- **Nova:** Compute service which orchestrates and schedules Virtual Machines
- **Glance:** Image Service for VM image storing
- **Cinder:** Block storage with volumes that can be attached to a VM
- **Horizon:** Dashboard UI for administration (can be seen set up on the local server in figure 4.1)
- **Neutron**: Networking service for managing networks, float IPs and security

All of the components above run as a process managed by the system under the snap with minimal manual interaction.

Comparing MicroStack Cloud deployment (moving our Gunicorn+Nginx bundle inside the MicroStack VM and using Cinder as a storage) with direct server setup (Gunicorn+Nginx) there are several easily noticeable advantages:

- **Isolation:** Each VM is sandboxed with its own kernel, network and storage
- **Scaling:** Duplicate VM instances
- **Centralized management:** Dashboard for orchestration
- **Logging:** Review API gateway logs and error reports
- **Role access control:** decide which users can access the dashboard and its parts

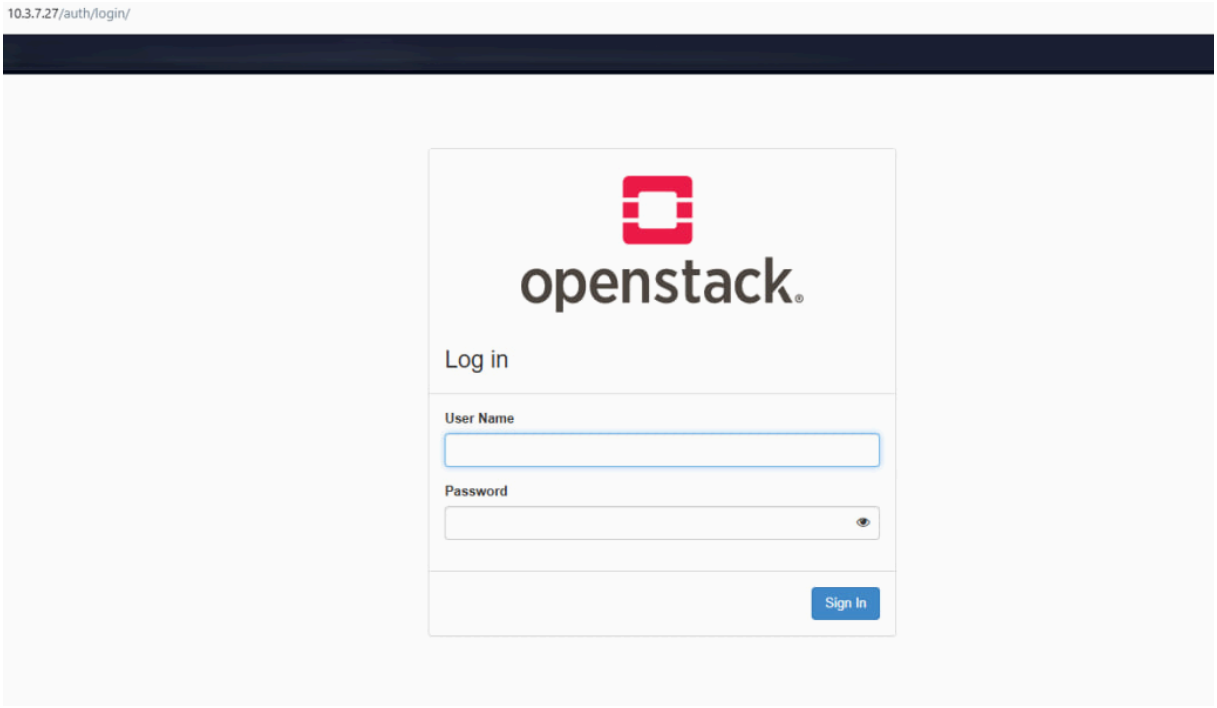

10.3.7.27/auth/login/
openstack.
Log in
User Name
Password
Sign In

Figure 4.1

5. Final summary

Over the course of this project, a Whisper's Tiny model was finetuned on 3 dataset splits resulting in 3 models capable of transcribing Hungarian, Russian and English speech. To make this process easier and personalized, all the finetuning and evaluating scripts were either modified or created from scratch. The results show that even one weak personal machine is capable of training a decently good model (33% average WER). On top of all that, Flask web application was developed implementing an elegant solution for dynamic memory for audio records and secure endpoints with user sessioning. In the end, the final version of the web application was deployed to the server, discussing two different approaches to do it: using MicroStack or raw deployment via Gunicorn and Nginx bundle. Altogether, this project laid a solid foundation for future development and scaling, not only in terms of improving the model, but also upgrading the functionality of the application itself.

Reference list

[1] S. Alharbi et al., "Automatic Speech Recognition: Systematic Literature Review," in IEEE Access, vol. 9, pp. 131858-131876, 2021, doi: 10.1109/ACCESS.2021.3112535.

[2] Statista. (n.d.). Statista - Bergur Thormundsson publications, 20 Apr. 2025, URL - https://www.statista.com/aboutus/our-research-commitment/3204/bergur-thormundsson

[3] Google Cloud. Speech-to-Text, 25. Apr. 2025, URL - https://cloud.google.com/speech-to-text

[4] Amazon. Amazon Transcribe, 25 Apr. 2025, URL - https://aws.amazon.com/transcribe/

[5] Microsoft. Azure AI Speech, 25 Apr. 2025, URL - https://azure.microsoft.com/en-us/products/ai-services/ai-speech

[6] OpenAI. Whisper, 1 May 2025, URL - https://openai.com/index/whisper/

[7] Korbinian Kuhn, Verena Kersken, Benedikt Reuter, Niklas Egger, and Gottfried Zimmermann. 2024. Measuring the Accuracy of Automatic Speech Recognition Solutions. ACM Trans. Access. Comput. 16, 4, Article 25, 2023, https://doi.org/10.1145/3636513

[8] Medium. David Cochard - Whisper: Speech Recognition Model Capable of Recognizing 99 Languages, 2023, URL - https://medium.com/axinc-ai/whisper-speech-recognition-model-capable-of-recognizing-99-languages-5b5cf0197c16

[9] Huggingface. About huggingface, 3 May 2025, URL - https://huggingface.co/huggingface

[10] Mozilla Foundation. Common Voice, 5 May 2025, URL - https://commonvoice.mozilla.org/en

[11] Huggingface. Launching Accelerate scripts, 5 May 2025, URL - https://huggingface.co/docs/accelerate/en/basic_tutorials/launch

[12] Dipina Damodaran B, Salim S, Vargese SM. Performance evaluation of MySQL and MongoDB databases. Int. J. Cybern. Inform.(IJCI). 2016; 5:387-94.

[13]Gunicorn. Documentation, 10 May 2025, URL - https://gunicorn.org/#docs

[14]Nginx. Documentation, 10 May 2025, URL - https://nginx.org/en/docs/

[15] Medium. Shafin Hasnat - MicroStack: Deploy OpenStack with Ease, 2023, URL - https://shafinhasnat97.medium.com/microstack-deploy-openstack-with-ease-b46391b684ae

Links

Link 1: GitHub repository of the project - https://github.com/SharDxD/speech-to-text-app

Statement

I, the undersigned Kirill Golubev, a student majoring in Computer Science MSc, hereby declare that I have prepared my thesis at the Department of Computer Science, Institute of Informatics, University of Szeged, for the purpose of obtaining my MSc degree in Computer Science.

I declare that I have not previously defended this thesis in another program, it is the result of my own work, and I have used only the cited resources (literature, tools, etc.). I am aware that my thesis will be stored in the University of Szeged Thesis Repository.

Date: 23.05.2025 Kirill Golubev

Signature: 

Acknowledgement